

The Art of Code Review

A Guide to Constructive Feedback

The Purpose of Code Review

Code review isn't about finding faults—it's about:

- Sharing knowledge
 - Catching bugs early
 - Maintaining code quality
 - Teaching and learning
 - Building team cohesion
-

What to Look For

Functionality

- Does it do what it's supposed to do?
- Are edge cases handled?

- Are error cases handled?
- Are there any obvious bugs?

Design

- Is the design consistent with the codebase?
- Are there better patterns available?
- Is it over-engineered or under-engineered?
- Will it scale?

Readability

- Is the code easy to understand?
- Are names clear and descriptive?
- Is the logic straightforward?
- Are comments helpful (not just noise)?

Testing

- Are there tests?
- Do the tests cover the new code?
- Are edge cases tested?
- Do the tests actually test something?

Performance

- Are there obvious performance issues?
 - Could it be optimized without sacrificing readability?
 - Are there memory leaks or resource issues?
-

How to Give Feedback

Be Specific

✗ "This is confusing"

✓ "This function name doesn't clearly indicate it also validates input"

Be Constructive

✗ "This is wrong"

✓ "Have you considered handling the null case here?"

Explain Why

✗ "Use a map instead"

✓ "A map would be more efficient here since we're doing lookups by key"

Ask Questions

✗ "You should refactor this"

✓ "What do you think about extracting this into a separate function?"

Point Out Good Things

✓ "I like how you handled the error case here"

✓ "This is a clever solution to the edge case"

How to Receive Feedback

Listen First

Don't get defensive. The reviewer is trying to help.

Ask for Clarification

If you don't understand a comment, ask. "Can you explain what you mean by X?"

Discuss, Don't Argue

If you disagree, explain your reasoning. But be open to being wrong.

Say Thank You

Code review takes time. Acknowledge the effort.

Common Patterns

The Nitpick

Small style issues. Usually safe to fix, but don't let them derail the review.

The Architecture Question

Bigger design questions. These are worth discussing in detail.

The Learning Opportunity

When the reviewer suggests something you didn't know. Great chance to learn!

The Trade-off

When there are multiple valid approaches. Discuss the trade-offs.

The Review Checklist

- ☐ Code does what it claims to do
 - ☐ Tests are present and meaningful
 - ☐ Code follows project conventions
 - ☐ No obvious bugs or security issues
 - ☐ Documentation is updated (if needed)
 - ☐ Performance is acceptable
 - ☐ Code is readable and maintainable
-

Remember

Code review is a conversation, not a judgment. The goal is better code, not perfect code. And better code comes from collaboration, not criticism.